icates in the same page of findings, typically because the scanners are alerting on the same vulnerability being described by two different CNAs. These can be considered as false positives, with the justification mentioning the duplicate finding identifier.

Repositories

A growing number of containers are being provided to and scanned. Knowing where to look to verify these findings and where to create an issue for them can sometimes be tricky. In some situations the individual repositories are the appropriate place (such as for gitlab-rails or gitaly). In other cases, a change needs to be made in the CNG repository or elsewhere.

SECTION: security/product-security/application-security/runbooks/hackerone-process.md

Purpose and Overview of GitLab's Bug Bounty Program

High-level description of the process

GitLab's HackerOne process manages vulnerability reports through a structured workflow where security researchers submit findings through HackerOne, which are then triaged by the HackerOne team before moving to GitLab's security team. The AppSec engineer on rotation assigns, validates, and imports valid reports into GitLab issues, calculating CVSS scores to determine severity and bounty amounts. They follow specific protocols for different vulnerability types (exposed secrets, vulnerability chaining, DNS takeovers), maintain regular communication with reporters, and ensure proper remediation tracking. After fixes are deployed, reports are closed and may be publicly disclosed following a 30-day waiting period, with successful reporters potentially earning both bounties and Ultimate licenses.

Key Stakeholders and Responsibilities

- HackerOne Triage Team
- GitLab AppSec Engineering
- PSIRT Security Program Manager
- GitLab Product Manager of the feature affected by the finding
- GitLab Engineering (Development) Manager of the feature affected by the finding
- SIRT (Security Operations)
- Infrastructure Team

HackerOne Workflow

- GitLab uses HackerOne for its bug bounty program where security researchers report vulnerabilities.
  - Notifications about report status changes go to the hackerone-feed Slack channel.
- Key queues in the process:
  - New: contains all reports in the New state
  - GitLab Team: validated reports awaiting assignment to team members
  - H1 Triage: reports being reviewed by HackerOne triage team
  - Pending Disclosure: reports ready for review and disclosure
- Guiding principles for handling reports:

  - Monitor H1 Triage queue for Critical or High reports
  - AppSec engineers on rotation should ensure proper assignment and triage
  - Reports can be reassigned to the next person on rotation if needed
- Working the queue process:
  - Use a dedicated namespace with Ultimate license for reproduction testing
  - Trust but verify the HackerOne Triage Team's validation
  - Assign reports to yourself when beginning work
  - Prioritize by severity, close duplicates, and triage oldest reports first
- Report validation workflow:
  - Review the validation performed by HackerOne
  - Communicate with reporters and investigate as needed
  - Determine if the report is out-of-scope, a feature, informative, or a duplicate
- For valid, in-scope reports:
  - Calculate CVSS score and set appropriate severity
  - Import the report into a GitLab issue
  - Assign proper labels, due dates, and notify relevant team members
  - Change report state to "Triaged" in HackerOne
- Special handling processes exist for:
  - Vulnerability chaining (multiple vulnerabilities in one report)
  - Exposed secrets (tokens, credentials)
  - Exposed personal data
  - Features behind feature flags
  - DNS record takeovers
- Awards and bounty process:
  - Partial awards may be given at triage time
  - Awards require approval based on severity level
  - After 30 days, approved awards may be paid before fixes are confirmed
- Issue management and disclosure:
  - Communicate regularly with reporters (at least monthly)
  - Follow SLA exception process when needed
  - Close and disclose issues after patches are released (30-day waiting period)
- Additional benefits:
  - Researchers with 3+ valid reports are eligible for 1-year Ultimate licenses
  - HackerOne Triage Team members receive Ultimate licenses

HackerOne Report Lifecycle Timeline

TBD

Step-by-Step Triage and Remediation Procedure

Queues

- New contains all reports in the New state
- GitLab Team contains reports that have been validated by the HackerOne triage team, but are yet to be assigned to a specific GitLab team member
- H1 Triage are reports being triaged by the HackerOne triage team
- Pending Disclosure are reports that should be reviewed and disclosed

Guiding principles

- When the GitLab Team queue is empty, regularly check that the H1 Triage queue doesn't contain reports that are rated as Critical or High. If there are such rated reports, evaluate if they are indeed Critical or High, and if so handle them directly without waiting on H1 Triage.
  - Generally speaking it's a good practice to keep an eye on the H1 Triage and New queues to look for Criticals and Highs.
- The AppSec engineer on rotation should make every effort to ensure that all H1 reports that are assigned to GitLab Team within their triage week are both assigned (to themselves) and properly triaged.
  - If a report wasn't re-assigned to the person on rotation, the next person on rotation can freely assign it to them.

GitLab Team On-boarding

- New members of the GitLab security team are granted access to the GitLab HackerOne team with an access request issue using the appropriate role based entitlement template, which should be submitted by their manager during onboarding
- During onboarding, new GitLab security team members will be invited to join the HackerOne program if their role requires it.

Working the Queue

Namespace with Ultimate license for triaging

Please ensure that you use this namespace to create projects and groups required for testing vulnerabilities. This namespace is dedicated to reproduction of HackerOne issues.

HackerOne Triage Team

GitLab's bug bounty program is managed by HackerOne. The HackerOne triage team are the first responders, and will work with researchers to validate reports before assigning to GitLab Team.

We usually trust the HackerOne Triage Team and don't necessarily validate the report a second time. There are however cases when the GitLab team member on rotation may want to re-validate it, for example (non exhaustive list):

- There may be additional impact that require more investigation
- The severity can't be properly assessed without further investigation

GitLab Team

- When beginning work on a report, the security team member should assign the report to themselves immediately.
- It is OK to take a report that you will not work on immediately, especially if it is a duplicate or related to another report you are familiar with, just be sure to get it reassigned if you won't be able to meet the estimated triage time.
- When starting a triage work cycle, team members should prioritize as follows:
  1. Identify, triage, and escalate any New severity::1/priority::1 issues first, from any queue.
  1. Close duplicate and invalid reports.
  1. Triage further using Sort by "Oldest" reports.

1. Triage the GitLab Team queue.

If a reporter has questions about the order of report responses, 06 - Duplicates/Invalid Reports Triaged First common response may be used.
- Review the validation performed by the HackerOne triage team
- Communicate with the reporter and investigate, using the following guidelines as necessary:
  - Request clarification, from either the reporter or the triage team
  - Verify the report yourself
- When a report contains externally-hosted static content for reproduction (for example some HTML file triggering a CSRF or a vulnerability exploiting a postMessage issue), follow the instructions in this project to re-host it internally
- Potential, non-bounty outcomes:
  - Report is out-of-scope. If actionable, issues may still be created.
  - Report is a ~"type::feature" as defined above and would not need to be made confidential or scheduled for remediation. An issue can be created, or requested that the reporter creates one if desired, but the report can be closed as "Informative".
  - Report is otherwise Informative, Not Applicable, or Spam. For example, we've gotten a number of reports for inline HTML in markdown that do not exceed the capabilities of markdown itself.
  - Report is a duplicate. Verify that the issue was not previously reported in HackerOne, or that an issue does not already exist in GitLab. If it is a duplicate:
    - Change the state of the report to "Duplicate".
    - If the issue was previously reported in H1, include the report id, as it can impact the reporter's reputation
    - Fill in the 01 - Duplicate common response. Include the link to the GitLab issue.
    - The team member may use their discretion if the reporter asks to be added as a contributor to the original H1 report; however, the default is to not add because the corresponding GitLab issue will be made public 30 days after the patch is released. If it is decided to add the duplicate reporter, ensure that the report does not have reporter sensitive information before allowing access.
      05 - Duplicate Follow Up, No Adding Contributors/Future Public Release common response can be used when denying the request to add as a contributor.
    - If the new report makes us realize we have mishandled a previous report (e.g. closed as informative but it was a valid bug) we reopen the original and award the new report a bounty to thank the reporter for putting the mishandled issue back on our radar
      - The bounty we award as a thank you should be $100 for low severity reports and equivalent to the initial bounty we pay on triage for the higher severities
      - If we realize our mistake late in the process and we have already rewarded the new report, the bounty should be raised to the amounts above or left as is if it's already higher
      - The author of the duplicate report should also be thanked in the CVE credits and blog post
      - If the new report demonstrates new and higher impact, we calculate the CVSS score and award to the new reporter the difference between the new severity and what was awarded to the original report
- If the report relates to information disclosure, follow the triaging exposed secrets process.
- If the report is valid, in-scope, original, and requires action, security-related documentation change, or if the report needs further investigation by

the responsible engineering team:
  - Calculate the CVSS score and post the resulting vector string (e.g.: AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:L) as an internal comment on the report, this will be used later when requesting a CVE ID
  - Verify and/or set the appropriate Severity in H1, using the CVSS previously calculated
    - Optionally explain the CVSS to the researcher, mention that CVSS scores are validated by a peer, and link to our Awards process to avoid inefficient misunderstandings relating to severity and payouts
  - Verify and/or set the appropriate Weakness in H1
  - If the report is permissions related, check for similar issues in the API, GraphQL, and Elasticsearch, as appropriate. Also check with alternate authentication mechanisms like Deploy Tokens, Deploy Keys, Trigger Tokens, etc.
  - Add your initial suggested bounty in H1
  - Import the report into a GitLab issue using /h1 import <report> [project] [options] in Slack
    - Note: by default a placeholder CVE issue is created and a brief note is added to the latest bug bounty council issue. Pass ~no-cve or ~no-bounty respectively to the /h1 import command to prevent their creation.
  - On the imported GitLab issue:
    - Verify the Severity/Priority assigned by h1import (Severity and Priority and Remediation SLAs)
    - Assign the appropriate Due Date
    - Have a proper How to reproduce section, by for instance copying the final reproduction steps written by our HackerOne triager into the issue.
    - If the report is a security-related documentation change, add the ~documentation label
    - @-mention the product manager and engineering manager based on the product categories page. Ask for engineering feedback if it is required to complete the triage
    - add labels (/label ~ command) corresponding to the DevOps stage and source group (consult the Hierarchy for an overview on categories forming the hierarchy)
    - As applicable, notify other relevant team members through the issue, chat, and email, depending on the chosen security level.
  - Change the state of the report to "Triaged" in HackerOne:
    - See GitLab's H1 Policy, under Rewards, for portions of bounty rewards which are awarded at the time of triage
    - Choose from the following common responses:
      - 00 - Triaged pending further investigation for reports where the severity or validity is uncertain, and we are discussing with the engineering team.
      - 00 - Triaged for low severity reports, which do not have an initial bounty at the time of triage
      - 00 - Triaged with Bounty for medium, high, and critical reports which do have an initial bounty at time of triage
    - In the comment, include link to the confidential issue
  - Update the CVE issue and Bug Bounty Council note with relevant details, while they are still fresh in your mind
    - If the CVSS score is higher on GitLab.com than self-managed, calculate both scores and share them in the Bug Bounty Council issue. If the council agrees that security impact is higher on GitLab.com than self-managed, bounty award will be based on the CVSS for GitLab.com. The CVE and security release blog post will always use the self-managed CVSS.
    - Consider using the "Public description" field in the bug bounty council note. You can use the Duo-generated Public description that was created automatically if it is relevant and does

not reveal too many details. This field will be picked up by the CVE description update automation and used as the CVE description if present.
  - If you relied on the HackerOne Triage Team's validation of the issue, consider setting time in your calendar to validate it yourself. This will help if you need to validate the fix later.
  - If full impact is needed to be assessed against GitLab infrastructure, instead of testing in https://gitlab.com, use https://staging.gitlab.com/help to sign in with your GitLab email account
    - If multiple users are needed, use credentials for users gitlab-qa-user stored in 1password Team Vault to access the staging environment
  - If the report is for a bug that can be detected without authentication (in GitLab or anything else we host) consider reaching out to the Red Team or Vulnerability Management to help create a Nuclei template that we can include into our scanning
- Remember to review the Pending Disclosure tab and follow our disclosure process

 Calculating CVSS for Vulnerability Chaining

Typically, each HackerOne report discloses a single vulnerability. However, sometimes a single report uses two or more newly discovered vulnerabilities chained together for increased impact.

When a single report discloses two or more new vulnerabilities being chained together for greater impact, calculate the CVSS for each individual vulnerability and an additional "vulnerability chaining" CVSS score for the combined impact. Share both the individual CVSS scores and the "vulnerability chaining" CVSS score in the corresponding bug bounty council issue.

The CVSS of each individual vulnerability will be used for the CVEs issued for each vulnerability. The "vulnerability chaining" CVSS will be used to determine bounty award for the report.

For future reports that involve "vulnerability chaining" with previously disclosed vulnerabilities, only calculate the CVSS for newly disclosed vulnerabilities. In such cases, only the new vulnerabilities in the chain will be eligible for a "vulnerability chaining" CVSS-based bounty.

 Triaging exposed secrets

<details>
<summary>Click to view copy-pasteable Appsec Triage Checklist markdown to help with triaging exposed secrets</summary>

markdown
 Appsec Triage checklist
- [ ] Mitigate the incident if possible
  - [ ] If the exposed secret is a Agent Token:
    -] Validate if the token is a valid one by following the steps [here and gather the output for the SIRT incident.
    -] [Reset the token and reach out to the owner of the token through Slack DM and in the SIRT issue that you will create in the steps below.
  - [ ] If the exposed secret is a Personal Access Token:
    -] Using the API, gather the output of [/api/v4/user and

/api/v4/personalaccesstokens/self for the SIRT incident.
    -] [Revoke the token and reach out to the owner of the token through Slack DM and in the SIRT issue that you will create in the steps below.

  -] Post a comment in security-revocation-self-service using [this message template
  - [ ] If the information was leaked in an issue, make the Issue confidential and leave an internal note explaining why it's been made confidential.
-] Use the /security slack command to [initiate an incident
  - [ ] In the description section, include a link to the HackerOne report and any other useful information
    - [ ] Share the reporter's IP address(es) and time(s) the reporter accessed the sensitive data to assist with incident response.
  - [ ] In the remediation section, document what time and from what IP used to revoke the token or validate the leak.
  - [ ] If in doubt, choose "Urgent".
  - [ ] If you've been able to mitigate the incident, choosing "Not Urgent" is fine. The SEOC typically responds quickly anyway.
- [ ] Add information to the SIRT issue.
  - [ ] Update the Timeline section to include the date the secret was leaked, when HackerOne report came in, and when you took any actions.
  - [ ] Add any comments to the SIRT issue with context or information that might be helpful.
- [ ] In the H1 report use the reference field to link to the SIRT issue (for example if the incident issue is https://gitlab.com/gitlab-sirt/incidentXXXX/-/issues/1 the reference should be gitlab-sirt/incidentXXXX/-/issues/1)
-] Identify the most appropriate [non-CVSS bounty amount and add your initial suggested bounty in H1
- [ ] Use /h1 bounty REPORTID to create a comment on the Bug Bounty Council issue (this step should not be necessary if /h1 import was previously run without the ~no-bounty option.)
-] Support SIRT as required and, if applicable, follow the process for [handling severity::1/priority::1 issues
- [ ] Investigate the location of the exposure, and locations like it, for further exposure.
  - [ ] Check the history on issue / MR descriptions
  - [ ] Check git commit history
  - [ ] Check build artifacts
  - [ ] Use Advanced Search to look for similar patterns in other projects used by GitLab team members
-] If it was leaked through GitLab Unfiltered, add the report to the [tracking issue

</details>

Exposure of information and secrets is handled a little differently to vulnerabilities, as there is nothing to patch and therefore no need for a GitLab Project Issue, CVSS, or CVE. When a leak occurs:

- Mitigate the incident if possible
  - If the exposed secret is a Agent Token:
    - Validate if the token is a valid one by following the steps here and gather the output

for the SIRT incident.
    - Reset the token and reach out to the owner of the token through Slack DM and in the SIRT issue that you will create in the steps below.
  - If the exposed secret is a Personal Access Token:
    - Using the API, gather the output of /api/v4/user and /api/v4/personalaccesstokens/self for the SIRT incident.

      bash
      curl -H "Authorization: Bearer LEAKEDTOKEN" https://gitlab.com/api/v4/user
      curl -H "Authorization: Bearer LEAKEDTOKEN" https://gitlab.com/api/v4/personalaccesstokens/self

    - Revoke the token and reach out to the owner of the token through Slack DM and in the SIRT issue that you will create in the steps below.

        bash
        curl --request DELETE -H "Authorization: Bearer LEAKEDTOKEN" https://gitlab.com/api/v4/personalaccesstokens/self

  - Post a comment in security-revocation-self-service using this message template
  - If the information was leaked in an issue, make the Issue confidential and leave an internal note explaining why it's been made confidential.
- Use the /security slack command to initiate an incident
  - Learn more about engaging the SEOC: <https://handbook.gitlab.com/handbook/security/security-operations/sirt/engaging-security-on-call/engage-the-security-engineer-on-call>
  - In the description section, include a link to the HackerOne report and any other useful information
    - Share the reporter's IP address(es) and time(s) the reporter accessed the sensitive data to assist with incident response.
  - In the remediation section, document what time and from what IP used to revoke the token or validate the leak.
  - If in doubt, choose "Urgent".
  - If you've been able to mitigate the incident, choosing "Not Urgent" is fine. The SEOC typically responds quickly anyway.
- Add information to the SIRT issue.
  - Update the Timeline section to include the date the secret was leaked, when HackerOne report came in, and when you took any actions.
  - Add any comments to the SIRT issue with context or information that might be helpful.
- In the H1 report use the reference field to link to the SIRT issue (for example if the incident issue is https://gitlab.com/gitlab-sirt/incidentXXXX/-/issues/1 the reference should be gitlab-sirt/incidentXXXX/-/issues/1)
- Identify the most appropriate non-CVSS bounty amount and add your initial suggested bounty in H1
- Use /h1 bounty REPORTID to create a comment on the Bug Bounty Council issue (this step should not be necessary if /h1 import was previously run without the ~no-bounty option.
- Support SIRT as required and, if applicable, follow the process for handling severity::1/priority::1 issues

- Investigate the location of the exposure, and locations like it, for further exposure.
  - Check the history on issue / MR descriptions
  - Check git commit history
  - Check build artifacts
  - Use Advanced Search to look for similar patterns in other projects used by GitLab team members
- If it was leaked through GitLab Unfiltered, add the report to the tracking issue

Triaging exposed personal data

Similar to how we handle exposed secrets, we sometimes handle exposed personal data which also doesn't need a GitLab Project Issue, CVSS or CVE.

- Mitigate the incident if possible
  - If the information was leaked in an issue, make the Issue confidential and leave an internal note explaining why it's been made confidential.
  - :warning: Bear in mind that turning an issue confidential doesn't turn attachments confidential.
- Use the /security slack command to initiate an incident
  - Learn more about engaging the SEOC: <https://handbook.gitlab.com/handbook/security/security-operations/sirt/engaging-security-on-call/engage-the-security-engineer-on-call]>
  - Pick "Information Disclosure" as the nature of incident
  - In the description section, include a link to the HackerOne report and any other useful information
    - If possible, share the reporter's IP address(es) and time(s) the reporter accessed the sensitive data to assist with incident response.
  - "Not Urgent" is usually fine for these kind of leaks. The SEOC typically responds quickly anyway.
- Add information to the SIRT issue.
  - Add any comments to the SIRT issue with context or information that might be helpful.
- In the H1 report use the reference field to link to the SIRT issue (for example if the incident issue is https://gitlab.com/gitlab-sirt/incidentXXXX/-/issues/1 the reference should be gitlab-sirt/incidentXXXX/-/issues/1
- Identify the most appropriate non-CVSS bounty amount and add your initial suggested bounty in H1
- Use /h1 bounty REPORTID to create a comment on the Bug Bounty Council issue
  - Note that we are importing it using bounty only here.
- Support SIRT as required and, if applicable, follow the process for handling severity::1/priority::1 issues
- Investigate the location of the exposure, and locations like it, for further exposure.
  - Check the history on issue / MR descriptions
  - Use Advanced Search to look for similar patterns in other projects used by GitLab team members

Triaging features behind a feature flag

Sometimes researchers will report a vulnerability in features behind a feature flag. These reports are excellent as they allow us to patch vulnerabilities prior to them affecting our wider audience that utilizes the default settings. These reports are eligible for the full

amount of their calculated bounty.

Pay attention to the full report to determine the Attack Complexity. The word complex in the bullet points below is as defined in the section 2.1.2 Attack Complexity in CVSS 3.1 Specification. Keep in mind, the aforementioned section says the following under the 2.1.2 Attack Complexity section - "If a specific reasonable configuration is required for an attack to succeed, the Base metrics should be scored assuming the vulnerable component is in that configuration.".

- A vulnerability in a feature behind a feature flag that is not complex will be paid out at AC:L (this is after assuming the feature flag is enabled on a vulnerable instance). However we will handle the report as if it's AC:H for triage and SLOs.
- A vulnerability in a feature behind a feature flag that is quite complex will still be AC:H (this is after assuming the feature flag is enabled on a vulnerable instance)

Vulnerabilities behind disabled-by-default feature flags do not need a CVE (use ~no-cve when importing) as they are patched in regular releases, not security releases.

Triaging issues in lower Ruby versions

Some vulnerabilities will only work on certain Ruby versions. In order to reproduce them locally using GDK, here is how you can change your Ruby version:

1. Update the Ruby version inside the following file to the required version:
   - gitlab-development-kit/.tool-versions
   - gitlab-development-kit/gitlab/.tool-versions
1. Run asdf install ruby <required-version> while inside the GDK directory.
1. Run gem install gitlab-development-kit while inside the GDK directory.
1. Go into the ./gitlab directory inside the GDK directory, and run bundle install.
1. Verify the Ruby version after running gdk restart and going to https://127.0.0.1:3000/admin

Triaging deprecated features

Vulnerabilities in deprecated features are triaged normally. See discussion for more information.

Triaging DNS record takeovers

DNS record takeovers typically require multiple teams in order to triage. The workflow is slightly different:

- Instead of pinging the team responsible for the given page (or service, in the case of MX or TXT records) we collaborate with SIRT and the SRE Oncall
- We import the HackerOne report to the infrastructure repository with /h1 import $REPORT infrastructure
- Engage SIRT with /security in Slack. This will allow SIRT to perform their investigatory duties related to this type of attack.
- Engage @sre-oncall in Slack. This notifies the SRE (but does not initiate a PagerDuty ping) on-call of a situation requiring their attention. In the relevant SIRT issue, the responder should be added to the issue by the GitLab SIRT.

Remediation of this vulnerability happens within the SIRT issue and typically involves deleting the dangling CNAME record. For issues involving MX record takeovers we typically work with our MX SaaS vendor, Mailgun to obtain control of the record. More information on MX record takeovers can be found here.

Awards

- See GitLab's H1 Policy, under Rewards, for portions of bounty rewards which are awarded at the time of triage
  - It is OK to delay awarding at time of triage. Examples are when we aren't sure if a report is intended behavior, or if it will be a documentation change. Remember to return and make a partial award if/when appropriate.
  - It is OK to have awarded a partial bounty at time of triage and later learn we have overpaid due to an adjustment of validity or severity
- If the report does not already have a council issue, use the /h1 bounty <report> Slack bot to post a note on the current ~"bug bounty council" issue
  - Add descriptions, similar issues, and other commentary as appropriate
- Award approval:
  - For documentation updates, an award can be paid immediately
  - For Low and Medium CVSS, an award can be paid after at least one team member reacts with a thumbsup emoji
  - For High and Critical CVSS, two team members must react with a thumbsup emoji on the note
  - If needed, make a request in sec-appsec when your note has not received enough votes
- After Bug Bounty Council approval:
  - Consider explaining the CVSS score and council discussion points on the canonical issue. This helps teams understand the severity during remediation and, when the issue becomes public, it gives transparency to researchers and customers on how we reached a given score.
  - If 30 days have passed since the issue was triaged, the approved award may be paid in advance of a confirmed fix using the 04 - Bounty Award / Reviewed and Awarded Prior to Fix common response.
  - Once a fix is shipped, award the remaining amount (or full amount if none was awarded at time of triage) using the 02 - Bounty Award common response
  - Add a · emoji to the bug bounty council thread after paying the bounty award.

We can award GitLab swag to reporters who have submitted a quality report that did not qualify for a monetary reward in our Bug Bounty program. To award swag, please follow the swag nomination process that is managed by our Developer Relations team.

We can also request to plant trees in the GitLab Forest as a non-monetary rewards, where reporters for various reasons can't accept swag or a monetary award. The amount varies based on the severity, please add a note in the request form based on the list here (Internal link).

Managing issues

Discussion and remediation of vulnerabilities can sometimes take longer than we would prefer. Even so, frequent communication with reporters is far better than leaving reporters in the dark, even if our progress is slow. Therefore:

- Once the corresponding GitLab issue has been assigned a milestone, an automated process will add a comment to the H1 report with the planned

fix date. If the issue has not been assigned to a milestone within 1 week of the product manager being @ mentioned on the issue, follow up with the product management team.
- In the case where fixes are not as clear or discussion is still on-going as to whether a patch will be created at all, reporters should be notified of updates at least monthly.
- In any case, no report should go "stale" where updates are not provided within the last month.

SLA exceptions

The HackerOne bot will automatically assign the correct due date based on severity of the imported issue. However, sometimes the issues may for various reasons not be patched within that timeframe. When this happens, development teams should open a SLA exception and have it approved by the Vulnerability Management team. The Application Security team is available to assist by providing guidance on these exception requests, but the expectation is that development teams will submit these requests and provide the justification and exception type.

Closing out & disclosing issues

When a patch is released and the award process complete, it is time to close the HackerOne issue.

After 30 days, follow the process for disclosing security issues.
Once this has occurred, the HackerOne issue can also be publicly disclosed on a case-by-case basis, following the same process to remove sensitive information.
We should not disclose, or request to disclose, a HackerOne issue while the GitLab issue remains confidential.

Comments made by GitLab Security Bot (@gitlab-securitybot) can be redacted by AppSec or SecAuto team members using the /h1 redact <commenturl> Slack command.

To create a HackerOne Hacktivity page which will help other researchers learn more about quality reporting we welcome disclosure of resolved reports which are unique and interesting.

If a researcher requests public disclosure of a closed non-resolved report (e.g. Informative or Not Applicable), we opt to cancel disclosure requests using the 08 - Canceled Disclosure Message template. Reporters should instead consider opening a public GitLab issue as this is the best way to raise and address non-vulnerability issues.

If a researcher insists on disclosure on HackerOne we should agree to disclose it regardless of quality unless there is a good reason not to.

Application Security Engineer Procedures for severity::1/priority::1 Issues

Please see Handling severity::1/priority::1 Issues

Closing reports as Informative, Not Applicable, or Spam

If the report does not pose a security risk to GitLab or GitLab users it can be closed without opening an issue on GitLab.com.

When this happens inform the researcher why it is not a vulnerability. It is up to the discretion of the Security Engineer whether to close the report as "Informative", "Not Applicable", or "Spam". HackerOne offers the following guidelines on each of these statuses:

- Informative: Report contained useful information but did not warrant an immediate action.
- Not Applicable: Report was invalid, ineligible, or irrelevant.
- Spam: Report is not a legitimate attempt to describe a security issue.

We mostly use the "Informative" status for reports with little to no security impact to GitLab and "Not Applicable" for reports that show a poor understanding of security concepts or reports that are clearly out of scope. "Spam" results in the maximum penalty to the researcher's reputation and should only be used in obvious cases of abuse.

It may be appropriate to suggest opening a public GitLab Issue for reproducible bugs that are not vulnerabilities.

If a Report is Unclear

If a report is unclear, or the reviewer has any questions about the validity of the finding or how it can be exploited, now is the time to ask. Move the report to the "Needs More Info" state until the researcher has provided all the information necessary to determine the validity and impact of the finding. Use your best judgement to determine whether it makes sense to open a confidential issue anyway, noting in it that you are seeking more information from the reporter. When in doubt, err on the side of opening the issue.

One the report has been clarified, follow the "regular flow" described above.

Breakdown of Effective Communication

Sometimes there will be a breakdown in effective communication with a reporter. While this could happen for multiple reasons, it is important that further communication follows GitLab's Guidelines for Effective and Responsible Communication. If communication with a reporter has gotten to this point, the following steps should be taken to help meet this goal.

- Take a step back from the situation. Do not respond immediately.
- Seek advice from the H1 triage engineer or, if you are more comfortable, a manager on how best to move forward.
- Consider writing the response multiple times, and have the response reviewed by the person you consulted.
- Once diffused, if the general situation is not currently documented, open an MR to the handbook on how it can be handled in the future.

If the situation leads to a code of conduct violation, follow the process for addressing Code of Conduct violations.

Addressing Rules of Engagement or Code of Conduct violations

When behavior violates our Rules of Engagement or HackerOne's Code of Conduct we use the Bug Bounty Council to discuss, agree on, and document our response. Add a comment to the current Bug Bounty Council Issue using the template found in the issue description.

In line with our Transparency value, we should try to explain to the researcher why we've taken action and what those actions were. However in some instances (e.g. program bans) it may be appropriate to let HackerOne handle all communication, to keep our team members safe from potential abuse or retribution.

Reports potentially affecting third parties

When GitLab receives reports, through HackerOne or other means, which might affect third parties the reporter will be encouraged to report the vulnerabilities upstream. On a case-by-case basis, e.g. for urgent or critical issues, GitLab might proactively report security issues upstream while being transparent to the reporter and making sure the original reporter will be credited. GitLab team members however will not attempt to re-apply unique techniques observed from bug bounty related submissions towards unrelated third parties which might be affected.

Vulnerabilities on third-party software are accepted according to the following rules, as stated in our HackerOne policy:
The report includes a new vulnerability, for which a patch is not available, or

- A patch has been available for more than 30 days.
- It has a clear and working proof of concept that illustrates the impact to GitLab.
- It has Critical or High impact to GitLab.

This does not include websites of third party software and services and only includes dependencies & packaged software.

HackerOne process FAQ and troubleshooting

What should I do if I accidentally import a H1 report?

If you accidentally import a H1 report that is not relevant or actionable (e.g., out-of-scope, informative, or a duplicate):

1. Close the resulting issue and leave a comment indicating the report was imported by mistake.
1. Close the associated CVE request issue, also noting that it was created in error.
1. In the relevant HackerOne bug bounty council thread, note the accidental import, add an · to the automatically created comment, and resolve the thread.

How do we handle situations where a CVE issue was automatically created but no CVE will be issued?

If a CVE request issue is automatically created but a CVE will not be issued:

- Add a comment to the CVE request issue explaining why a CVE will not be issued.
- Close the CVE request issue.

What happens if I import a duplicate H1 report?

- Close the issue and any related CVE request or bug bounty council threads.
- Comment that you are closing the issues and bug bounty council threads because the report is a duplicate.

What if I cannot reproduce a HackerOne report?

Ideally, we should only involve the product team once we've validated and reproduced the reported vulnerability.
If AppSec cannot reproduce an issue, it is unlikely that the product team will be able to either. Without steps to reliably reproduce a vulnerability, it will be difficult to determine the root cause, propose a meaningful fix, or verify whether a solution will fully address the issue.

- Reach out to the reporter or H1 triager who validated the report if the steps to reproduce the issue are unclear or lacking.
- Use your best judgment.
- Ask for help from your peers in situations where a reported vulnerability seems particularly severe or impactful, but you are having difficulty reproducing it.

Which H1 reports are not publicly disclosed?

We disclose vulnerabilities that affect the GitLab product and for which a CVE is issued.

Reports of problems which cannot be resolved by making changes to GitLab software product source code and where no CVE (with a CVSS) is being issued are typically not publicly disclosed.

What should I do if a reporter asks about another report in a different thread?

Our HackerOne policy states that:

> The only appropriate place to inquire about a HackerOne report's status is on the report itself. Please refrain from submitting your report or inquiring about its status through additional channels including any other unrelated HackerOne report, as this unnecessarily binds resources in the security team.

As such, if a reporter inquires uses a report to inquire about the status of a different report, respond using the 51 - Reminder for asking about other reports template under "Common Responses".

Awarding Ultimate Licenses

GitLab reporters with 3 or more valid reports are eligible for a 1-year Ultimate license for up to 5 users. As per the H1 policy, reporters will request the license through a comment on one of their issues. When a reporter has requested a license, the following steps should be taken:

1. Validate that the three reports were valid. That means they are Triaged or Resolved.
1. Validate that the three reports have not been used to obtain a previous license.
1. If the reports are not valid, respond to the reporter on H1 explaining the reason the license is not being issued.

1. If the reports are valid, visit the GitLab Support Internal Requests page and use the GitLab Support Internal Requests for Global customers request option, then select Hacker One Reporter license for the internal request type.
    - For Contact Name use the reporter's full name if available, otherwise their H1 handle
    - For Contact Email use the reporter's [username]@wearehackerone.com email address
1. Enter the associated license information in the H1 License Award sheet
1. Reply to the report on H1 use the 20 - Ultimate License Creation template.

The license will be sent to the reporter by CustomersDot. If the reporter claims that the license has not arrived, the app can be used to resend the license.
When that happens, the creation of a new license should be avoided.

Questions?

Members of the public can ask questions about our HackerOne bug bounty program here: https://gitlab.com/gitlab-com/gl-security/product-security/appsec/hackerone-questions/. Note that this repository is not the place to discuss or disclose reports and vulnerabilities.

HackerOne Triage Team GitLab licenses

All members of the HackerOne triage team have access to GitLab Ultimate licenses. HackerOne will inform us annually when the license needs to be renewed.

- Visit the GitLab Support Internal Requests page
- Use the GitLab Support Internal Requests for Global customers request option, then select Hacker One Reporter license for the internal request type.
- Paste the following:

text
Two new licenses are needed for our HackerOne Triage+ team. This team helps us triage HackerOne reports and will need at least two licenses with 50 users each. They need ultimate licenses to validate reports affecting any feature of the product.

License 1: analysts@managed.hackerone.com

License 2: analysts+1@managed.hackerone.com

Company: HackerOne Inc.

License type: Ultimate

Number of users for each license: 50

License duration: 1 year

SECTION: security/product-security/application-security/runbooks/handling-s1p1.md

The following process is a supplement to the first few steps of the critical release process

Once a potential severity::1/priority::1 issue is made known. The appsec engineer steps are as follows:

Triage

1. Triage and verify the issue as you normally would triage a report.
1. Finalize the CVSS score of the security issue with team member votes on Bug Bounty Council (BBC) thread before engaging the SIRT team. Consider using a sync call or Slack for the discussion due to time sensitivity. Capture the outcome of the discussion in the BBC thread. If a sync call or a Slack discussion was not possible due to AppSec team members in the region being on PTO or timezone issues, trigger the SIRT workflow if 4 hours have passed since the issue was triaged.
1. Within the BBC thread, create a GitLab Dedicated specific CVSS score.
1. To help SecOps quickly determine impact and log analysis, comment in the security issue with the summarized reproduction steps (HTTP Requests, generated log messages, images, etc).
1. After escalating, do an investigation to try to determine if there are other immediately vulnerable components or other impacts.

Escalate

1. Engage the Security Engineer on-call with a link to the issue, a summary of what has happened, and an description of what SIRT may need to do.
1. Engage the appropriate engineering manager and product manager of the affected component in both the issue and in the appropriate Slack channels.
1. If help from the GitLab Dedicated team is needed, follow the runbook to escalate to their engineer on call.
1. Ping @appsec-leadership in the sec-appsec Slack channel with a link to the issue. This will help team leadership and other engineers get up to speed, in case they need to step in.
1. Create a link to the Bug Bounty Council CVSS discussion in the SIRT incident GitLab issue.
1. Create a bookmark to the CVSS discussion in the incident specific Slack channel.

Evaluate Impact in Different Environments

Due to differences in settings, feature availability, and configuration between GitLab self-managed, GitLab Dedicated, and GitLab SaaS (GitLab.com), the CVSS for a single vulnerability may differ depending on environment.

To accurately communicate and effectively mitigate negative impact of a security vulnerability, consider any possible discrepancies in settings, feature availability, and configuration for the different environments - especially when there are certain preconditions required for a successful attack to occur.

GitLab Dedicated

When assessing if a GitLab vulnerability impacts GitLab Dedicated, consider the following features that are not available in GitLab Dedicated:

Application Features that are Unavailable in GitLab Dedicated

- [ ] LDAP, Smartcard, or Kerberos authentication

- [ ] Multiple login providers
- [ ] Advanced search
- [ ] GitLab Pages
- [ ] Reply-by email
- [ ] Service Desk
- [ ] FortiAuthenticator, or FortiToken 2FA
- [ ] GitLab-managed runners (hosted runners)
-] GitLab AI capabilities ([More Info)
-] Features that must be configured outside of the GitLab user interface, including those behind [feature flags which are disabled-by-default
- [ ] Mattermost
-] Server-side Git hooks (Due to security concerns and potential service SLA impact. Consider using [push rules or webhooks as alternatives.)

If a vulnerability requires using features listed above for successful exploitation, it most likely does not impact GitLab Dedicated. Always cross-check with the specific details of the vulnerability to ensure accurate assessment.

Mitigate

Mitigation of critical security issues has to strike a balance between securing GitLab and our users as fast as possible and doing it in a reliable way that will not require another patch shortly after.
The patch will first be deployed to GitLab-managed environments (.com, Dedicated, etc.) and then will be released to our self-managed users.

1. Collaborate with all the relevant teams (development team owning the feature, infrastructure, SIRT, etc.) to come up with a solution for the vulnerability.
1. Analyze the impact for each option.
    - How effective is it at solving the problem?
    - Are we patching a symptom or are we fixing the root cause?
    - How many customers are affected by this decision?
    - How exactly are they affected?
    - What's the magnitude?
    - What other positive and negative consequences are there?
1. Choose the solution that best balances the concerns above with the concerns of participating teams.
1. Once the solution has been delivered, validate that the fix was effective.

Occasionally, we'll need a quick fix before a good patch can be thoroughly developed and reviewed.
Here are some examples of short term options we've used in the past:

- Cloudflare rule to block certain endpoints.
- Disable a specific feature using feature flags or application configuration.
- Deploy a hotpatch.

Releasing to self-managed customers

Since moving to a bi-weekly release schedule the need to follow the critical security releases

workflow to patch critical vulnerabilities is much lower.
To decide between including a patch in the regular patch release or a critical security release many factors have to be considered.
Those factors include but are not limited to:

- How easily can this vulnerability be exploited?
- Does the vulnerability require a user account?
- Is the vulnerability being exploited in the wild?
- Is the exploit automatable and easily exploitable at scale?
- How impactful is the vulnerability?

This is handled on a case by case basis and will need to be evaluated with all the stake holders each time the situation arises.
Some scenarios where we would be very likely to opt for an ad-hoc critical security release would be:

- Unpatched critical severity vuln (CVSS 9+) being exploited in the wild
- Unpatched critical severity vuln (CVSS 9+) with PoC/exploitation code publicly available
- CVSS 10.0 vulns (e.g. unauth RCE or admin account takeover)

Initiate RCA

If it's a product vulnerability, the AppSec DRI must initiate a root cause analysis (RCA) investigation issue as soon as possible. It is incredibly important that the underlying root cause of the vulnerability is well-understood and documented in order to prevent bypasses or similar incidents. Followup preventative and mitigative control issues will be created and prioritized as a result of this step.

Handoff

Appsec engineers are not on-call. That means when the assigned appsec engineer's end of day arrives, they are responsible for handing it off to an appsec engineer in a subsequent timezone.

Provide a brief summary of the current status and outstanding or upcoming tasks required of AppSec. Provide useful links like the SIRT issue, comms document, slack links. Ideally also schedule a short synchronous call with the person to whom you're handing over, to discuss and answer questions.

Share that a handover has happened in the incident's Slack channel, and cross-post to other relevant channels like sec-appsec. A message template like the following may be appropriate:

> · AppSec Handover · I have handed over to @username for any AppSec needs, as I am close to the end of my working day. [Include details on how we will continue to deliver on any tasks that AppSec is DRI for].

After the incident

Apply the correct labels and milestones in the SIRT issue so that we can track the work done in our metrics.

text
/label ~"AppSecWorkType::SIRTandSecurityComms" ~AppSecWeight::<> ~"Application Security Team" ~"AppSecWorkflow::complete"
/milestone %<>

Family and Friends Day Coverage

Family and Friends Days are days where GitLab publicly shuts down.
There will be one AppSec engineer covering for each timezone region (AMER, EMEA, APAC) during each F&F Day.
See Holiday Coverage for more information.

SECTION: security/product-security/application-security/runbooks/holiday-coverage.md

This runbook describes the process for times when the Application Security team has team members available during holidays, Friends and Family days, or other events where many team members are expected to be out of office.

Since Application Security is not on-call, we aim to provide this best-effort coverage that may or may not be available in every region.

If you need assistance:

- Post in sec-appsec with details
- Find the AppSec engineer providing coverage - see Communicating the coverage below
- Reach out to the team member via Slack @ mention
- If urgent or the team member does not respond in a timely manner, reach out via text message or phone call after checking the contact preferences listed on their Slack profile
- If it is unclear who is providing coverage, ping the @appsec-team in Slack

Expectations for those doing coverage

- In the event that AppSec is needed to respond to an S1 incident or otherwise urgently assist another team, the team member providing coverage should be reachable by Slack or phone
- AppSec team members providing coverage are not expected to be at the keyboard and working, but they should be able to get online within one hour if needed
- If they are not actively working during coverage times, they should try to check Slack occasionally for pings
- They should occasionally check for potential S1s coming in through HackerOne and take action if it requires immediate attention
- If an S1 incident occurs or AppSec assistance is urgently needed by another team, the expectation is that the AppSec team member will attempt to triage the situation and escalate as needed to a security manager

Planning

When holidays or Friends and Family days are coming up, the AppSec team should try to find a

team member who can swap their day off. Ideally a team member for each region would be available, but it is not necessarily required.

Use the Rotation Management issue tracker to assign & coordinate coverage.

Communicating the coverage

AppSec's Triage Rotation handbook page has information on the rotations and who is assigned.

The AppSec team will post a Slack message in sec-appsec with information on who is available, when they are available, and how to get in contact if AppSec assistance is urgently needed. This should be cross-posted to te security and security-division channels for extra visibility.

Each AppSec team member providing coverage will have their mobile phone number available in their Slack profile.

Should an incident occur, the "Handling S1/P1s procedure" will be followed, which includes handing over to the next AppSec engineer.

SECTION: security/product-security/application-security/runbooks/investigating-package-hunter-findings.md

List of Package Hunter Findings

Any Package Hunter related finding can be found on this dashboard (internal link).

Network Connection Findings

We're going to use this finding as an example. In this finding,
Package Hunter detected that a package opened a network connection.

> 02:24:56.163600570: Notice Disallowed outbound connection destination (command=node scripts/install.js connection=172.17.0.2:36884->52.217.109.44:443 user=root containerid=fb38d63ef02a containername=some-container-eae8d3ec-a482-441b-8262-78198b46fdfb.tgz image=maldep)

Investigation

- We can look at the destination IP address and see if gives any interesting information
  - In this example, it's a an AWS IP address but it gives us no information about what package might have contacted this IP
- Attention: This step requires to checkout the branch for which the finding was made. Please proceed with caution when analyzing potentially malicious code on your local computer. We recommend to checkout the code into a dedicated VM. If there are any questions or you require assistance, please reach out to @gitlab-com/gl-security/product-security/appsec.
  - We know that the package ran the command scripts/install.js so we can search for that file (be sure to execute yarn install before running the search)

  - Running find . -name install.js leads us to nodemodules/node-sass/scripts/install.js
- We can inspect the package.json files to find the command that triggered the detection
  - We see in nodemodules/node-sass/package.json that scripts/install.js is indeed called

Completing the investigation in the example, we see that the install.js script
calls getBinaryURL(). This function is defined in nodemodules/node-sass/lib/extensions.js:

```
javascript
/
  Determine the URL to fetch binary file from.
  By default fetch from the node-sass distribution
  site on GitHub.
 /
function getBinaryUrl() {
  var site = getArgument('--sass-binary-site') ||
            process.env.SASSBINARYSITE  ||
            process.env.npmconfigsassbinarysite ||
            (pkg.nodeSassConfig && pkg.nodeSassConfig.binarySite) ||
            'https://github.com/sass/node-sass/releases/download';  return [site, 'v' +
pkg.version, getBinaryName()].join('/');
}
```

We see from this file that node-sass is trying to download the binary
from https://github.com/sass/node-sass/releases/download/VERSION/BINARYNAME

Doing this locally we obtain:

```
shell
$ wget https://github.com/sass/node-sass/releases/download/v5.0.0/linux-x64-64binding.node
...
Connecting to github-production-release-asset-2e65be.s3.amazonaws.com (github-production-
release-asset-2e65be.s3.amazonaws.com)|52.216.204.107|:443... connected.
...
```

It's resolving the URL to 52.216.204.107, which is not exactly the same IP
from the Package Hunter finding, but IP address info is very similar and it's
normal that the request wouldn't exactly be served by the same IP every time
in this context. At this point we can be reasonably sure about the source of
the alert and that there is no malicious intent.

Actions

- If we discover that the package was malicious all along (typosquatting for example),
[security on-call should be engaged] for further investigation and an MR should be
opened to replace the package with the legitimate one. Consider reporting
the malicious package to the registry operator (NPM or RubyGems)
- If we discover that a legitimate package was compromised,
security on-call should be engaged for further investigation and an MR should be

opened to roll back to a previous version of the package that is known to be secure.
Consider reporting the malicious package to the registry operator (NPM or RubyGems)
and to the maintainer of the package
- If we discover that the network connection is used to fetch a legitimate resource,
we should look into the possibility of self-hosting the resource or verifying its
integrity during the build process to protect against a potential compromise of
the external resource

SECTION: security/product-security/application-security/runbooks/jihu-contribution-merge-monitor-reports.md

The Merge Monitor tool looks in public GitLab repositories that JiHu contributes to for merge requests that:

- Were merged
- Were labeled as a JiHu Contribution
- Were not labeled with the label that AppSec team members need to apply after conducting security reviews of JiHu contributions

Any findings will be included in reports that are created as issues in the jihumergerequestmonitorreports repository. The Federal AppSec team is pinged on each report that is created and the expectation is that they will review each finding.

The Merge Monitor runs via scheduled pipeline. It de-duplicates any findings by checking for open Merge Monitor Report issues for related merge requests and filtering out any findings that are already known.

Merge Monitor Report Process

For each finding mentioned in a Merge Monitor report:

1. View the merge request and check to see if it received an AppSec review
    - If it received an AppSec review, apply the sec-planning::complete label to the merge request
    - If it did not receive an AppSec review, perform one
1. After verifying that a review was performed or performing a review, check the MR reviewed checkbox
1. After applying the sec-planning::complete label to the merge request, check the Label applied checkbox
1. For each finding, edit the issue description and write a brief summary and check the Summary checkbox
    - This should be simple, such as AppSec review was not performed before merge or Review was performed but sec-planning label did not get applied
    - If needed, add a comment to the issue to document reasons why the merge request did not receive a review, gaps in the process, and opportunities for improvement. Link to the comment on the Summary line.
1. Once each finding in the report has been reviewed, close the issue

In the event that you find a vulnerability or other security concern in a finding:

1. Triage it as normal by creating a new issue in the appropriate repository and apply priority/severity labels
1. Ping the merge request author, reviewers, and the AppSec team on the issue
1. If possible, work to get the merge request reverted before it is included in a release
    - If a release is in-progress, contact the delivery team to see if it can be removed from the release
    - If this was already included in a release and this is a S1 finding, follow the normal S1 process
1. Add a note to the Merge Monitor report that indicates a vulnerability was identified with a link to the issue that was created to track it

Monitor Limitations

Since the Merge Monitor uses a Project Access Token in the jihumergerequestmonitorreports, it can only be used to find merge requests in public repositories that the JiHu team contributes to. Some repositories require manual review as mentioned in the certification process documentation and are not covered by this tool. Contributions to these repositories are reviewed as part of the regular monthly release certification process.

SECTION: security/product-security/application-security/runbooks/local-setup.md

When evaluating security issues or MRs, it can be useful to have a way to reproduce issues, dig in to root causes, look for further impacts. This can also be a great way to get familiar with GitLab during your first few weeks of onboarding. Here are some handy tips & tricks.

Get a version of GitLab to play with

Many AppSec engineers will opt to install GitLab locally using GDK. Here's how to install GDK and how to use it. The UX team have a great summary of other ways to play with & test GitLab.

Step through execution chains

If you want to see the code executed as part of a web or API request, an interactive debugger may be a useful tool. Here's how to configure Pry & Thin

A typical workflow might be to find the Controller action which kicks off the request (methods like create or update are good bets), add in binding.pry, save the file, then perform that request in a browser. The execution will stop and in a terminal you can inspect the current state using IRB, type step to go into a method, next to go to the next statement, and continue to let the request run to the next break point and/or completion.

Watching logs can be helpful: tail -f gitlab/log/development.log.

Install a testing proxy

Your role might not require you to do "penetration testing", but having access to a testing

proxy that lets you intercept and manipulate requests can help with reproducing HackerOne issues.

The AppSec team have a multi-user license for Burp Suite Professional. Ask in sec-appsec about getting a license, and (download the latest stable version here). You can also use OWASP ZAP which is free and open source.

These tools can easily cause damage to websites or eat up your CPU with active scans. In OWASP Zap, use "Safe" mode to prevent any potentially malicious requests. In Burp Suite, disable any live "audit" scans.

Browser Profiles

When testing requires using multiple users, an Incognito / Private tab is an easy option. You can also create and use un-signed-in Chrome Profiles or Firefox Multi-Account Containers to provide "session sandboxes", which will persist beyond window closure (unlike Incognito tabs) and you can colour code them to help with visual distinction.

Mocking Servers / tunnels

Making your local machine accessible from the internet is not permitted, which precludes tools like ngrok or localtunnel. Use GitLab's Sandbox Cloud to host mock servers instead. Refer to Secure Cloud testing environments for advice on how to secure your Sandbox Cloud test environments.

SECTION: security/product-security/application-security/runbooks/review-process.md

:warning: Prioritization Note

As of 2023-11-02, AppSec is only prioritizing P1 AppSec reviews and threat models. This does not mean P2 or P3 AppSec reviews or threat model requests can't be submitted, but please understand that due to capacity limitations we will only be able to prioritize P1 reviews. Reach out to us in the sec-appsec Slack channel if you have any questions or concerns.

Review preparation

To start an actual review a few preparation steps need to be done in order to have a systematic and comprehensible approach amongst all reviewers. Main output of the preparation step is a scope definition and time estimation.

Scope definition

Based on the provided input in the review issue the reviewer should be able to build a prioritized list of sub-features and testable items which are in scope and to be tested within the review. Most of the time it might also be needed to define items/sub-features which will not be tested.

The reviewer needs to gather a high-level understanding of the feature to properly define the scope of the review. This means not only the amount of code and given time constraints need to be considered. The reviewer needs to have context of the environment the code will be executed in, which APIs are being provided, where inputs come from and which adjacent APIs are being called.

The in-scope list should be a rather high-level list, further refinement will be conducted in the subsequent review steps.

Prioritization

Different sub-features and functionalities of the feature need to be prioritized. For this apparent critical functionalities like authentication and authorization should get highest priority.

See Assigning Priority

The following items are considered high priority code parts:

- Parts which involve authentication:
  - Login functionality
  - Deploy keys or token
  - Password reset.
- Parts which involve authorization:
  - New access and permission checks of any kind
  - Modification of access rights and permissions
- File uploads
- Newly implemented security checks to prevent common vulnerabilities
- Introduction of new dependenci